

Authentification marchand — résumé d'implémentation

Vue d'ensemble du flux

```
Formulaire inscription → POST /api/marchands/inscription → Utilisateur(aktif=false) + Marchand(en_attente)
                        ↓
                        Admin approuve via PATCH /api/marchands/[id]/statut
                        ↓
                        POST /api/auth/login → cookie de session (JWT) → accès /marchand
```

Le principe existait déjà (RF-22 : un marchand auto-inscrit reste bloqué tant qu'un admin ne l'a pas validé). Le travail a consisté à étendre ce flux pour capturer tous les champs de la maquette, qui n'existaient pas encore dans le modèle de données.

Fichiers modifiés / créés

1. `prisma/schema.prisma` — modifié

Ajout de 7 champs sur le modèle `Marchand` pour couvrir les champs manquants de la maquette (CIN, site web, adresse, banque, photo RIB, ville de ramassage, registre de commerce) :

```
cin          String?
siteWeb      String? @map("site_web")
adresse      String?
nomBanque    String? @map("nom_banque")
ribPhotoUrl  String? @map("rib_photo_url")
villeRamassage String? @map("ville_ramassage")
registreCommerce String? @map("registre_commerce")
```

Les champs déjà existants (`nomBoutique` , `ville` , `rib` , `typeCompte` avec l'enum `TypeCompteMarchand`) ont été réutilisés tels quels.

2. `prisma/migrations/20260725130000_marchand_inscription_champs/migration.sql` — nouveau

Migration SQL correspondante (`ALTER TABLE marchands ADD COLUMN ...`).

Point d'attention : appliquée manuellement (`prisma db execute + prisma migrate resolve --applied`) plutôt que via `prisma migrate dev` , car la base *shadow* de Prisma échoue à rejouer l'historique existant (conflit d'index entre deux anciennes migrations `refonte_statuts_colis` déjà présentes avant cette intervention — un problème préexistant, sans lien avec ce chantier, à corriger séparément si besoin).

3. lib/marchand-form-options.ts — nouveau

Listes partagées entre le formulaire et l'API (pour éviter toute divergence) :

- `VILLES_RAMASSAGE` : villes marocaines pour le dropdown « Ville de ramassage »
- `BANQUES_MAROC` : banques marocaines pour le dropdown « Nom du banque »
- `TYPES_COMPTE` / `LABELS_TYPE_COMPTE` : basés sur l'enum Prisma `TypeCompteMarchand`

4. lib/types.ts — modifié

Interface `Marchand` (types front) complétée avec les 7 nouveaux champs + `typeCompte`, pour que le profil marchand (`/api/marchands/me`) reste correctement typé.

5. app/api/marchands/inscription/route.ts — réécrit

Endpoint public (déjà whitelisted dans `proxy.ts`, aucun changement nécessaire là) qui reçoit maintenant tous les champs du formulaire :

- **Champs requis** : `nomComplet`, `cin`, `nomBoutique`, `telephone`, `email`, `secret`, `ville`, `adresse`, `rib`, `ribPhotoUrl`
- **Champs optionnels** : `siteWeb`, `nomBanque`, `registreCommerce`, `villeRamassage`, `typeCompte`
- **Validations** : format email (regex), mot de passe ≥ 4 caractères, RIB = exactement 24 chiffres, unicité téléphone **et** email (le champ email existait déjà en base avec une contrainte `@unique` mais n'était pas exploité à l'inscription)
- Création transactionnelle `Utilisateur` (`actif=false`) + `Marchand` (`statut=en_attente_validation`), comme avant

6. app/inscription/page.tsx — réécrit

Formulaire complet reproduisant la maquette :

- Disposition 2 colonnes (responsive → 1 colonne sur mobile)
- Champs texte à soulignement (au lieu du style « carte noire » utilisé ailleurs dans l'appli, pour coller à la maquette fournie)
- Mot de passe / confirmation avec icône œil (afficher/masquer, `lucide-react`)
- RIB : saisie filtrée aux chiffres, limitée à 24 caractères
- Photo RIB : convertie en *data URL* côté navigateur (`FileReader`) et envoyée telle quelle — même pattern que les preuves de livraison déjà stockées dans l'appli (pas de stockage objet dédié)
- Dropdowns Banque / Type d'entreprise / Ville de ramassage
- Bouton « Envoyer la demande » en dégradé, lien « Sign in » vers `/login`
- Vérification côté client que mot de passe = confirmation avant envoi

7. app/globals.css — modifié

Deux nouvelles classes utilitaires ajoutées dans `@layer components` pour reproduire le style de la maquette :

- `.input-underline` : champ à bordure basse uniquement

- `.btn-gradient` : bouton pilule en dégradé bleu → violet

Fichiers non modifiés mais qui font partie du flux (pour référence)

Fichier	Rôle
<code>app/login/page.tsx</code>	Formulaire de connexion (déjà fonctionnel, redirige vers <code>/marchand</code> selon le rôle)
<code>app/api/auth/login/route.ts</code>	Vérifie téléphone + mot de passe, refuse si <code>actif=false</code>
<code>lib/auth.ts</code>	Hash bcrypt, signature/vérification JWT, cookie de session
<code>proxy.ts</code>	Protège toutes les routes <code>/api/**</code> sauf login et inscription
<code>app/api/marchands/[id]/statut/route.ts</code>	Endpoint admin pour approuver un marchand

Comment réviser / tester

1. **Relire le diff** : tous ces fichiers sont non-suivis par git pour l'instant (`git status` les montre en `??`), donc un `git add + git diff --cached` donne une vue complète.
2. **Lancer l'appli** : `npm run dev` , puis `http://localhost:3000/inscription` .
3. **Tester le flux complet** :
 - Remplir le formulaire → devrait afficher « Compte créé, en attente de validation... »
 - Se connecter avec `0000000000` / `1234` (compte admin du seed) → aller sur `/admin/marchands` → approuver le nouveau marchand
 - Se reconnecter avec le téléphone/mot de passe du marchand → doit rediriger vers `/marchand`
4. **Cas d'erreur à vérifier** : téléphone/email déjà utilisé (409), RIB pas 24 chiffres (400), mots de passe qui ne correspondent pas (bloqué côté client).

Limites connues

- **Pas de test visuel réel en navigateur** : aucun outil Playwright/Chromium n'était disponible dans l'environnement de développement lors de l'implémentation, donc le rendu exact (alignement, couleurs) n'a pas été vérifié à l'œil — seulement le HTML généré et le bon fonctionnement de l'API.
- **Page admin marchands non mise à jour** : elle liste toujours les marchands mais n'affiche pas encore CIN, photo RIB, etc. À faire si besoin pour la validation.
- Un serveur `next dev` déjà lancé avant l'intervention a dû être arrêté (PID périmé avec un client Prisma obsolète) pour pouvoir tester — relancer `npm run dev` si besoin.